

3 - Ciclos

Alejandro Veloz

aavelozb.github.io/py



Objetivos de la unidad

- Resolver problemas con ciclos de una **cantidad predefinida de iteraciones** (`while` + contador).
- Resolver problemas con ciclos controlados por **condiciones generales** (usando *banderas* cuando haga falta).
- Reconocer y aplicar los **patrones** clásicos: contar, acumular, buscar el mayor/menor, buscar algo.
- Escribir **ciclos anidados**: repetir un proceso que ya es repetitivo.
- Dar **seguimiento (ruteo)** a un programa con ciclos.

Objetivos de la unidad

- Resolver problemas con ciclos de una **cantidad predefinida de iteraciones** (`while` + contador).
- Resolver problemas con ciclos controlados por **condiciones generales** (usando *banderas* cuando haga falta).
- Reconocer y aplicar los **patrones** clásicos: contar, acumular, buscar el mayor/menor, buscar algo.
- Escribir **ciclos anidados**: repetir un proceso que ya es repetitivo.
- Dar **seguimiento (ruteo)** a un programa con ciclos.

Todo se hace con `while`.

En esta unidad **no** usamos `for`, ni `break`, ni `continue`.

El problema

Imprimir los números del 1 al 5:

```
print(1)
```

```
print(2)
```

```
print(3)
```

```
print(4)
```

```
print(5)
```

El problema

Imprimir los números del 1 al 5: Ahora del 1 al 1000...

```
print(1)  
print(2)  
print(3)  
print(4)  
print(5)
```

¿Copiar y pegar 1000 líneas?

El problema

Imprimir los números del 1 al 5: Ahora del 1 al 1000...

```
print(1)
print(2)
print(3)
print(4)
print(5)
```

¿Copiar y pegar 1000 líneas?

Y peor: si el usuario elige n , el número de líneas **no se conoce** al escribir el programa.

El problema

Imprimir los números del 1 al 5: Ahora del 1 al 1000...

```
print(1)
print(2)
print(3)
print(4)
print(5)
```

¿Copiar y pegar 1000 líneas?

Y peor: si el usuario elige n , el número de líneas **no se conoce** al escribir el programa.

Necesitamos una instrucción que **repita**.

La instrucción while

La instrucción while

Repite un grupo de instrucciones **mientras** una condición sea True.

```
while condicion:  
    instruccion 1  
    instruccion 2  
    ...  
# resto del programa
```

La instrucción while

Repite un grupo de instrucciones **mientras** una condición sea True.

```
while condicion:  
    instruccion 1  
    instruccion 2  
    ...  
# resto del programa
```

- La condición se evalúa **antes** de cada repetición.
- Si es True, se ejecuta **todo** el bloque indentado (una *iteración*) y se vuelve a evaluar.
- Si es False, el ciclo termina y el programa sigue después del bloque.

La instrucción while

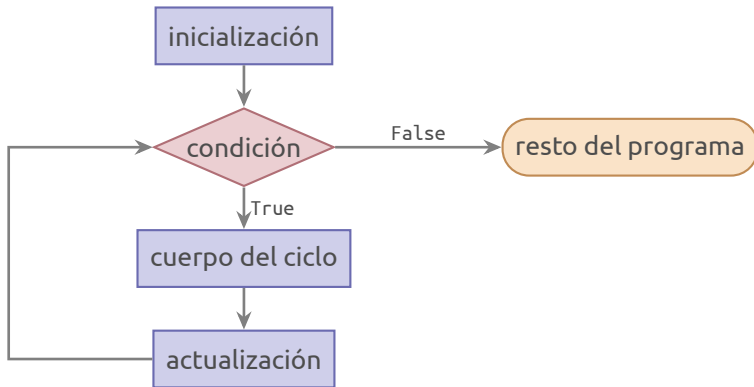
Repite un grupo de instrucciones **mientras** una condición sea True.

```
while condicion:  
    instruccion 1  
    instruccion 2  
    ...  
# resto del programa
```

- La condición se evalúa **antes** de cada repetición.
- Si es True, se ejecuta **todo** el bloque indentado (una *iteración*) y se vuelve a evaluar.
- Si es False, el ciclo termina y el programa sigue después del bloque.

Si la condición es falsa la primera vez, el cuerpo no se ejecuta ninguna vez.

La instrucción while



Anatomía de un ciclo

Casi todos los ciclos que escribiremos tienen estas cuatro partes:

<code>i = 1</code>	# 1. inicialización
<code>while i <= 5:</code>	# 2. condición de continuidad
<code>print(i)</code>	# 3. cuerpo (lo que se repite)
<code>i += 1</code>	# 4. actualización

Anatomía de un ciclo

Casi todos los ciclos que escribiremos tienen estas cuatro partes:

```
i = 1           # 1. inicialización
while i <= 5:   # 2. condición de continuidad
    print(i)    # 3. cuerpo (lo que se repite)
    i += 1      # 4. actualización
```

Parte	Si falta...
inicialización	error: la variable no existe
condición	error de sintaxis
cuerpo	el ciclo no hace nada útil
actualización	ciclo infinito

Recordatorio: operadores de asignación abreviada

```
i += 2    # i = i + 2
i -= 1    # i = i - 1
i *= 5    # i = i * 5
i //= 2   # i = i // 2
```

Son **atajos de escritura**: primero se calcula el lado derecho y el resultado se guarda en la misma variable.

Los usaremos todo el tiempo para **contar** (`i += 1`) y para **acumular** (`suma += num`).

¿Cuántas veces se ejecuta?

```
i = 0  
n = 4  
while i < n:  
    print(i)
```

¿Cuántas veces se ejecuta?

```
i = 0  
n = 4  
while i < n:  
    print(i)
```

Infinitas veces.

i nunca cambia, así que $i < n$ siempre es True.

¿Cuántas veces se ejecuta?

```
i = 0  
n = 4  
while i < n:  
    print(i)  
    i += 1
```

¿Cuántas veces se ejecuta?

```
i = 0
n = 4
while i < n:
    print(i)
    i += 1
```

4 veces: 0, 1, 2, 3.

Cuenta de 0 a $n-1$.

¿Cuántas veces se ejecuta?

```
i = 1  
n = 4  
while i <= n:  
    print(i)  
    i += 1
```

¿Cuántas veces se ejecuta?

```
i = 1
n = 4
while i <= n:
    print(i)
    i += 1
```

4 veces: 1, 2, 3, 4.

Cuenta de 1 a n. Ojo con el <=.

¿Cuántas veces se ejecuta?

```
i = 1  
n = 4  
while i < n:  
    print(i)  
    i += 1
```

¿Cuántas veces se ejecuta?

```
i = 1  
n = 4  
while i < n:  
    print(i)  
    i += 1
```

3 veces: 1, 2, 3.

Condiciones de borde: cambiar <
por <= cambia la cantidad de
iteraciones.

¿Cuántas veces se ejecuta?

```
i = 0  
n = 4  
while i <= n:  
    print(i)  
    i += 1
```

¿Cuántas veces se ejecuta?

```
i = 0
n = 4
while i <= n:
    print(i)
i += 1
```

Infinitas veces.

La actualización quedó **fuera** del ciclo: la indentación decide qué se repite.

Ciclo infinito

Ocurre cuando la condición de continuidad **nunca** llega a ser falsa.

```
i = 1
while i != 100:
    print(i)
    i += 2
```

Ciclo infinito

Ocurre cuando la condición de continuidad **nunca** llega a ser falsa.

i toma los valores 1, 3, 5, ...

Nunca vale exactamente 100.

```
i = 1
while i != 100:
    print(i)
    i += 2
```

Ciclo infinito

Ocurre cuando la condición de continuidad **nunca** llega a ser falsa.

```
i = 1
while i != 100:
    print(i)
    i += 2
```

i toma los valores 1, 3, 5, ...

Nunca vale exactamente 100.

- Si ocurre al ejecutar, se detiene con **Ctrl + C**.
- Regla práctica: prefiera **<** o **<=** en vez de **!=** cuando la variable avanza de a saltos.

Un ciclo puede no ejecutarse nunca

Patrón muy útil: **validar la entrada**.

```
n = int(input('n: '))  
while n <= 0:  
    print('Debe ser positivo')  
    n = int(input('n: '))
```

Un ciclo puede no ejecutarse nunca

Patrón muy útil: **validar la entrada**.

```
n = int(input('n: '))  
while n <= 0:  
    print('Debe ser positivo')  
    n = int(input('n: '))
```

- Si el usuario ingresa 5 a la primera, el cuerpo **no se ejecuta**.
- Si ingresa -3, se le vuelve a preguntar **tantas veces como sea necesario**.

Un ciclo puede no ejecutarse nunca

Patrón muy útil: **validar la entrada**.

```
n = int(input('n: '))  
while n <= 0:  
    print('Debe ser positivo')  
    n = int(input('n: '))
```

- Si el usuario ingresa 5 a la primera, el cuerpo **no se ejecuta**.
- Si ingresa -3, se le vuelve a preguntar **tantas veces como sea necesario**.

La cantidad de iteraciones no siempre se conoce de antemano.

Patrones con ciclos

Los patrones

#	Patrón	Idea
1	Iterar n veces	contador de 0 a $n-1$, o de 1 a n
2	Iterar hasta un valor centinela	leer hasta que aparezca cierto valor
3	Iterar hasta un umbral	detenerse cuando algo se cumple
4	Banderas (<i>flags</i>)	una variable <code>bool</code> controla la salida
5	Conteo	contador $+= 1$
6	Mayor / menor	comparar contra el mejor hasta ahora
7	Acumulación	suma $+= x$ / producto $*= x$
8	Promedio	conteo + acumulación
9	Ciclo anidado	repetir algo que ya es repetitivo

1. Iterar n veces

Contando de 0 a n-1

```
n = int(input('Cantidad de veces: '))  
i = 0  
while i < n:  
    print(i)  
    i += 1
```

1. Iterar n veces

Contando de 1 a n

```
n = int(input('Cantidad de veces: '))  
i = 1  
while i <= n:  
    print(i)  
    i += 1
```

Ambos hacen n iteraciones. Elija el que haga más natural el cuerpo del ciclo.

2. Iterar hasta que se ingrese cierto valor

```
n = int(input('Ingrese número (0 para terminar): '))  
while n != 0:  
    n = int(input('Ingrese número (0 para terminar): '))
```

- El valor que termina el ciclo se llama **centinela** (aquí, el 0).
- Note las **dos lecturas**: una antes del ciclo (para poder evaluar la condición) y otra al final del cuerpo.

3. Iterar hasta que se cumpla una condición

```
suma = 0
while suma < 100:
    n = int(input('Ingrese número: '))
    suma += n
```

La condición no cuenta iteraciones: observa el **estado** del programa. No sabemos cuántos números se pedirán.

4. Uso de banderas para terminar un ciclo

```
n = int(input("Ingrese número a evaluar: "))
primo = True
divisor = 2

while primo and divisor <= n-1:
    if n % divisor == 0:
        primo = False
        divisor += 1

if primo:
    print(n, ' es primo')
else:
    print(n, ' es compuesto')
```

4. Uso de banderas para terminar un ciclo

- Una **bandera** (*flag*) es una variable `bool` que resume “algo pasó”.
- Sirve para **salir apenas se sabe la respuesta**, sin recorrer el resto.
- Use nombres que se lean solos: `primo`, `encontrado`, `continuar`.

4. Uso de banderas para terminar un ciclo

- Una **bandera** (*flag*) es una variable bool que resume “algo pasó”.
- Sirve para **salir apenas se sabe la respuesta**, sin recorrer el resto.
- Use nombres que se lean solos: primo, encontrado, continuar.

Estructura general:

```
continuar = True
while continuar and <otra condicion>:
    ...
    if <ya sé la respuesta>:
        continuar = False
```

Así evitamos usar break.

5. Patrón de conteo

```
n= int(input('Cantidad de números: '))
contador = 0
i = 0
while i < n:
    if i % 2 == 0:
        contador += 1
    i += 1
print (contador, 'pares')
```

Dos variables distintas: *i* avanza el ciclo, contador cuenta los casos que interesan.

6. Patrón para encontrar el mayor

```
n= int(input('Cantidad de personas: '))
mayor = 0 # Alternativamente mayor float('-Inf')
i = 1
while i <= n:
    edad = int(input('Edad: '))
    if edad > mayor:
        mayor = edad
    i += 1
print('La mayor edad es:', mayor)
```

6. Patrón para encontrar el mayor

```
n = int(input('Cantidad de personas: '))
mayor = int(input('Edad: ')) # Leer el primero
                             # antes del ciclo

i = 1
while i < n:
    edad = int(input('Edad: '))
    if edad > mayor:
        mayor = edad
    i += 1
print ('La mayor edad es:', mayor)
```

7. Patrón para encontrar el menor

```
n = int(input('Cantidad de personas: '))
menor = 1000 # Alternativamente mayor float('Inf')
i = 1
while i <= n:
    edad = int(input('Edad: '))
    if edad < menor:
        menor = edad
    i += 1
print('La menor edad es:', menor)
```

7. Patrón para encontrar el menor

```
n = int(input('Cantidad de personas: '))
menor = int(input('Edad: ')) # Leer el primero
                             # antes del ciclo

i = 1
while i < n:
    edad = int(input('Edad: '))
    if edad < menor:
        menor = edad
    i += 1
print('La menor edad es:', menor)
```

8. Patrón de acumulación (suma)

```
n = int(input('n: '))
suma = 0
i = 1
while i <= n:
    suma += i
    i += 1
print('La suma es:', suma)
```

El acumulador se inicializa **antes del ciclo**, y en el neutro de la operación: 0 para sumar.

8. Patrón de acumulación (multiplicación)

```
n = int(input('n: '))
factorial = 1
i = 1
while i <= n:
    factorial *= i
    i += 1
print('El factorial de', n, 'es:', factorial)
```

Para multiplicar, el neutro es 1. Si se inicializa en 0, el resultado siempre es 0.

9. Patrón para calcular promedio

(combinación de los patrones de conteo y suma)

```
num = int(input('Ingrese número (0 para terminar): '))
suma = num
n = 1
while num != 0:
    num = int(input('Ingrese número (0 para terminar): '))
    if num != 0:
        suma += num
        n += 1
promedio = suma / n
print(promedio)
```

Ruteo

Ruteo de un programa con ciclos

Seguir la ejecución **a mano**, anotando cada cambio de valor.

- Hay una **columna para cada variable** que usa el programa, y una para la **pantalla**.
- Se agrega una **fila sólo cuando hay una asignación**.
- En una fila debe haber **un solo valor**.
- En la columna de pantalla se anota **todo** lo que el programa imprime.

Ruteo de un programa con ciclos

Seguir la ejecución **a mano**, anotando cada cambio de valor.

- Hay una **columna para cada variable** que usa el programa, y una para la **pantalla**.
- Se agrega una **fila sólo cuando hay una asignación**.
- En una fila debe haber **un solo valor**.
- En la columna de pantalla se anota **todo** lo que el programa imprime.

Rutear es la forma más rápida de encontrar un error de borde.

Ruteo: conjetura de Collatz

```
t = 13
while t != 1:
    print(t)
    if t % 2 == 0:
        t = t // 2
    else:
        t = 3*t + 1
print(1)
```

Complete la secuencia que se imprime:

13 __ 20 __ __ __ 8 4 2 1

Ruteo: conjetura de Collatz

```
t = 13
while t != 1:
    print(t)
    if t % 2 == 0:
        t = t // 2
    else:
        t = 3*t + 1
print(1)
```

Complete la secuencia que se imprime:

13 __ 20 __ __ __ 8 4 2 1

13 40 20 10 5 16 8 4 2 1

Ruteo: ejercicio

```
n = int(input('n: '))
f0 = 1
f1 = 1
i = 2
while i <= n:
    fn = f0 + f1
    f0 = f1
    f1 = fn
    i += 1
print(f1)
```

Rutee el programa para $n = 6$.

n	f0	f1	fn	i

Errores frecuentes

Encuentre el error

Imprimir los números impares de 1 a 100.

```
i = 1
while i != 100:
    print(i)
    i += 2
```

Encuentre el error

Imprimir los números impares de 1 a 100.

```
i = 1
while i != 100:
    print(i)
    i += 2
```

Ciclo infinito: i es siempre impar, nunca vale 100.

```
i = 1
while i <= 100:
    print(i)
    i += 2
```

Encuentre el error

Contar cuántos pares hay en los 50 números leídos como entrada.

```
i = 1
contador = 0
while i <= 50:
    num = int(input('Ingrese un número: '))
    if num % 2 == 0:
        contador += 1
i += 1
```

Encuentre el error

Contar cuántos pares hay en los 50 números leídos como entrada.

```
i = 1
contador = 0
while i <= 50:
    num = int(input('Ingrese un número: '))
    if num % 2 == 0:
        contador += 1
i += 1
```

La actualización `i += 1` quedó fuera del ciclo (ciclo infinito), y falta imprimir el resultado.

¡Ojo a la indentación!

Encuentre el error

Sumar 5 números ingresados como entrada e imprimir el resultado.

```
i = 1
while i <= 5:
    suma = 0
    num = int(input('Nro: '))
    i += 1
    suma += num
print(suma)
```

Encuentre el error

Sumar 5 números ingresados como entrada e imprimir el resultado.

```
i = 1
while i <= 5:
    suma = 0
    num = int(input('Nro: '))
    i += 1
    suma += num
print(suma)
```

suma = 0 está dentro del ciclo: se reinicia en cada iteración y al final sólo queda el último número.

El acumulador se inicializa antes del ciclo.

Encuentre el error

Sumar los números recibidos, hasta que se ingresa 0.

```
continuar = True
suma = 0
while continuar:
    n = int(input("Ingrese nro (0 para detener): "))
    if n == 0:
        suma += n
    else:
        continuar = False
```

Encuentre el error

Sumar los números recibidos, hasta que se ingresa 0.

```
continuar = True
suma = 0
while continuar:
    n = int(input("Ingrese nro (0 para detener): "))
    if n == 0:
        suma += n
    else:
        continuar = False
```

La condición del if está invertida: debe ser `if n != 0: suma += n` y `else:`
`continuar = False.`

Encuentre los errores

Sumar los números recibidos, hasta que se ingresa 0 o hasta que la suma sea 100 o más.

```
n = int(input("Ingrese nro (0 para detener): "))
meta = False
suma = 0
while n != 0 or meta == False:
    suma += n
    if suma >= 100:
        meta = True
    n = int(input("Ingrese nro (0 para detener): "))
```

Encuentre los errores

Sumar los números recibidos, hasta que se ingresa 0 o hasta que la suma sea 100 o más.

```
n = int(input("Ingrese nro (0 para detener): "))
meta = False
suma = 0
while n != 0 or meta == False:
    suma += n
    if suma >= 100:
        meta = True
    n = int(input("Ingrese nro (0 para detener): "))
```

(1) El or debe ser **and**: se sigue mientras *no* aparezca el 0 *y* *no* se llegue a la meta.

(2) La última lectura debe ir dentro de un **else**, para no pedir un número de más.

Ordene el código

Calcula el mayor, el menor y el promedio de un conjunto de edades. Las líneas están desordenadas y sin indentación.

```
1 while i <= n:
2 suma += edad
3 max = edad
4 i = 1
5 suma = 0
6 min = edad
7 prom = round(suma/n)
8 i += 1
```

```
9 if edad < min:
10 print(prom, max, min)
11 max = 0
12 edad = int(input('Edad: '))
13 n = int(input('Cantidad: '))
14 if edad > max:
15 min = 1000
```

Escriba el número de instrucción, en el orden e indentación correctos.

Ciclos anidados

Ciclos anidados

Un ciclo **dentro** del cuerpo de otro ciclo: sirve para repetir un proceso que **ya es repetitivo**.

```
i = 1
while i <= n:           # ciclo externo: n repeticiones
    j = 1               # <- reinicio del contador interno
    while j <= m:       # ciclo interno: m repeticiones
        ...             # se ejecuta n*m veces
        j += 1
    i += 1
```

Ciclos anidados

Un ciclo **dentro** del cuerpo de otro ciclo: sirve para repetir un proceso que **ya es repetitivo**.

```
i = 1
while i <= n:           # ciclo externo: n repeticiones
    j = 1               # <- reinicio del contador interno
    while j <= m:       # ciclo interno: m repeticiones
        ...            # se ejecuta n*m veces
        j += 1
    i += 1
```

Regla de oro: el contador del ciclo interno se **reinicia** dentro del ciclo externo, justo antes de entrar.

Tabla de multiplicar

Complete los espacios:

```
a = 5
b = 10
inicio = 1
fin = 1
while inicio <= ____:
    while fin <= ____:
        print(inicio, "x", fin, "=", inicio*fin)
        fin += ____
    fin = ____
    inicio += ____
```

Tabla de multiplicar

Complete los espacios:

```
a = 5
b = 10
inicio = 1
fin = 1
while inicio <= ____:
    while fin <= ____:
        print(inicio, "x", fin, "=", inicio*fin)
        fin += ____
    fin = ____
    inicio += ____
```

Si se olvida la línea `fin = 1`, el ciclo interno sólo se ejecuta la primera vez.

Primos en un rango

```
a = int(input("Ingrese limite inferior del rango: "))
b = int(input("Ingrese limite superior del rango: "))
candidato = a
while candidato <= b:
    # Evaluar si candidato es primo
    primo = True
    divisor = 2
    while primo and divisor <= candidato//2:
        if candidato%divisor == 0:
            primo = False
        divisor += 1
    if primo:
        print(candidato)
    candidato += 1
```

Dibujar una figura

Un triángulo de n filas: la fila i tiene i asteriscos.

```
n = int(input('n: '))
i = 1
while i <= n:
    linea = ''
    j = 1
    while j <= i:
        linea += '*'
        j += 1
    print(linea)
    i += 1
```

n: 5

```
*
**
***
****
*****
```

El ciclo interno **acumula** un string; el externo lo imprime.

Lo que usamos en esta unidad

SÍ

- while con contador.
- while con condición general.
- Variables *bandera* (bool).
- Patrones: contar, acumular, mayor/menor, buscar.
- Ciclos anidados.
- Ruteo de programas con ciclos.

NO

- break y continue: rompen el flujo natural del ciclo.
- for: se estudia en las unidades de *strings* y *listas*.

En vez de while True: ... if cond:
break, escribimos while not cond: ...

Ejercicios

Ejercicio: potencia

Complete el programa para que calcule la potencia `base ** exponente` sin usar `**`.

```
base = int(input('Base: '))
exp  = int(input('Exponente: '))
potencia = ____
i = ____
while i <= ____:
    potencia ____ base
    i += 1
print(potencia)
```

Ejercicio: potencia

Complete el programa para que calcule la potencia `base ** exponente` sin usar `**`.

```
base = int(input('Base: '))
exp  = int(input('Exponente: '))
potencia = ____
i = ____
while i <= ____:
    potencia ____ base
    i += 1
print(potencia)
```

(a) ¿se puede modificar este programa para calcular el factorial? **(b)** ¿qué cambios haría?

Ejercicio: validación

Modifique el programa de la potencia para evitar errores de ingreso:

1. Validar que la base sea distinta de 0; mientras no lo sea, volver a pedirla.
2. Validar que el exponente sea mayor o igual a 0; mientras no lo sea, volver a pedirlo.
3. **Bonus:** permitir exponentes negativos. ¿Qué cambios haría?

Ejercicio: Fibonacci

La serie de Fibonacci se define como:

$$f_0 = 1, \quad f_1 = 1, \quad f_n = f_{n-1} + f_{n-2}$$

es decir: 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Escriba un programa que lea la posición n y muestre el término correspondiente.

Ingresa posición: 5

8

Ingresa posición: 9

55

Ejercicio

Haga un programa que muestre los divisores de un número dado, los sume, y muestre el resultado de la suma en pantalla.

Ejercicio

Un número **perfecto** es un número natural igual a la suma de sus divisores propios (todos sus divisores, excepto él mismo).

$$28 = 1 + 2 + 4 + 7 + 14 \quad \text{es perfecto}$$

$$32 \neq 1 + 2 + 4 + 8 + 16 \quad \text{no es perfecto}$$

Escriba un programa que determine si un número es perfecto o no.

Ingrese un numero: 28

Es perfecto

Ingrese un numero: 32

No es perfecto

Ejercicio

Escriba un programa que lea dos números enteros y muestre la multiplicación entre ellos sin usar el operador `*` de Python.

Ejercicio

Escriba un programa que lea números enteros hasta que se ingrese el cero. En ese momento, muestra la multiplicación entre ellos (sin incluir el último) y termina.

Ingrese un numero: 4

Ingrese un numero: 7

Ingrese un numero: 1

Ingrese un numero: 0

28

Ejercicio

Escriba un programa que lea un número n e imprima los primeros n términos de la siguiente serie: 1, 2, 4, 7, 11, ... que comienza en 1 y se forma sumando 1, luego 2, luego 3, luego 4, y así sucesivamente.

Ejemplo:

Ingrese n : 10

1

2

4

7

11

16

22

29

37

46

Ejercicio

La famosa serie FizzBuzz para un número natural N es una sucesión desde 1 hasta N donde:

- Los números que sean múltiplos de 3 se cambian por Fizz.
- Los números que sean múltiplos de 5 se cambian por Buzz.
- Los números que sean múltiplos de 3 y 5 se cambian por FizzBuzz.

Escriba un programa que lea N e imprima la serie FizzBuzz.

```
Ingrese n: 15
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
```

En estadística se define el rango de un conjunto de datos como la diferencia entre el mayor y el menor de los datos.

Por ejemplo, si los datos son: 5.96 6.74 7.43 4.99 7.20 0.56 2.80, entonces el rango es $7.43 - 0.56 = 6.87$

Escriba un programa que:

- Pregunte al usuario cuántos datos serán ingresados.
- Pida al usuario ingresar los datos uno por uno.
- Entregue como resultado el rango de los datos.

Suponga que todos los datos ingresados son válidos.

Cuántos valores ingresará? 7

Valor 1: 5.96

Valor 2: 6.74

Valor 3: 7.43

Valor 4: 4.99

Valor 5: 7.20

Valor 6: 0.56

Valor 7: 2.80

El rango es 6.87

Ejercicio: raíz cuadrada

Un método para estimar la raíz cuadrada de n :

- Restarle a n el primer número impar (1), al resultado el siguiente impar (3), y así sucesivamente, hasta que el resultado sea menor o igual a cero.
- Si el resultado final es **cero**, la raíz es exacta y vale la cantidad de restas realizadas.
- Si es **menor que cero**, la raíz aproximada (truncada) es la cantidad de restas menos uno.

Ingrese un numero: 36

Raíz exacta: 6

Ingrese un numero: 8

Raíz aproximada: 2

Ejercicio: multiplicación rusa

Para multiplicar $a \times b$: mientras a sea mayor que 0, se divide a por 2 (división entera) y se duplica b ; se **acumulan** los valores de b en las filas donde a es **impar**.

a	b	
13	5	← suma
6	10	
3	20	← suma
1	40	← suma

$$5 + 20 + 40 = 65 = 13 \times 5$$

Escriba el programa que multiplica dos enteros positivos con este método, sin usar `*` en el cálculo del producto.

Un robot ha sido diseñado para moverse a lo largo de una cuadrícula, recibiendo como entrada alguna de las letras **N**, **S**, **E**, **O**, que le ordenan moverse un metro hacia el norte, sur, este, oeste, respectivamente. La letra **F** le pone fin al movimiento del robot.

Escriba un programa que simule el movimiento del robot, leyendo letras ingresadas una por una. Al finalizar el movimiento, debe imprimir la distancia recorrida y la distancia de la ruta óptima (camino más corto posible para llegar al mismo destino). Ejemplo:

```
Movimiento: N  
Movimiento: E  
Movimiento: E  
Movimiento: E  
Movimiento: S  
Movimiento: S
```

```
Movimiento: O  
Movimiento: O  
Movimiento: F  
Distancia recorrida: 8 m  
Distancia óptima: 2 m
```

Desafío: encuesta Deuman

Cada persona vota con un **número entero**, donde cada dígito es la nota (1 a 7) con que califica una canción que conoce. La calificación de la persona es el **promedio de sus dígitos**. Por ejemplo, el voto 234 corresponde a una calificación 3.0.

(a) Lea el voto de una persona y muestre su calificación, redondeada a 2 decimales.

```
Voto: 235  
3.33
```

(b) Lea votos hasta que se ingrese 0 y calcule la calificación máxima, mínima y promedio, además de cuántos votos se procesaron.

```
Voto: 235  
Voto: 22  
Voto: 323  
Voto: 0  
Cantidad de votos: 3  
Max: 3.33  
Min: 2.0  
Promedio: 2.67
```